

UART Transceiver Design & Verification Report

1. Project Introduction

1.1 Overview

The Universal Asynchronous Receiver–Transmitter (UART) is one of the oldest and most widely used serial communication protocols in digital systems. Unlike synchronous protocols (SPI, I²C), UART does not transmit a shared clock signal alongside data; instead, both communicating devices agree on a *baud rate* in advance and rely on that shared timing contract to frame every character correctly.

This project implements a **full-duplex UART transceiver** in synthesisable SystemVerilog. Full-duplex means the transmitter (TX) and receiver (RX) operate completely independently and simultaneously on separate physical lines, allowing a device to send and receive data at the same time.

1.2 Purpose of the Design

The design serves two purposes:

1. **Educational** — to demonstrate how a real-world serial protocol is decomposed into well-separated RTL modules (baud generator, transmitter FSM, receiver FSM) and how each interacts with the system clock domain.
2. **Functional** — to provide a synthesisable, parametrisable UART core that can be integrated into larger SoC or FPGA designs.

1.3 Main Functionality

- Accepts an 8-bit parallel data word, serialises it with a start bit and a stop bit, and transmits it on a single wire (TX line).
- Monitors a single wire (RX line), detects a start bit, deserialises the incoming bits by sampling at the mid-point of each bit period, and presents a recovered 8-bit parallel word to the host.
- Provides handshake signals (`xmit_active`, `xmit_doneH`, `rec_readyH`, `rec_busy`) so that a host processor can manage data flow without polling the raw serial line.
- Includes a two-flip-flop synchroniser on the RX input to guard against metastability from asynchronous external data.

2. Objectives

- Study the UART protocol frame format (start bit, data bits, stop bit) and understand how $16\times$ oversampling achieves robust mid-bit sampling.
- Implement a **parametrisable baud-rate generator** derived from a 100 MHz system clock, making the design portable to different FPGA platforms by changing a single header file.
- Implement a **synthesisable TX FSM** (`uxmit` module) with four states: `idle`, `start`, `trans`, `stop`.
- Implement a **synthesisable RX FSM** (`u_rec` module) with four states: `idle`, `start`, `trans`, `stop`, including false-start-bit detection.
- Compose the TX and RX sub-modules into a unified `uart` top-level wrapper that exposes a clean, host-facing interface.
- Create a directed, self-checking testbench covering all 16 planned test cases.
- Build a cycle-accurate FSM-mirror reference model (`uart_ref`) that replicates DUT logic exactly so that the scoreboard detects any behavioural deviation.
- Verify corner cases: asynchronous reset, reset during active transmission, all-zero / all-one data words, back-to-back frames, framing errors (missing stop bit), and `xmitH` held high for continuous transmission.
- Achieve 100 % functional coverage on all defined test cases.

3. Design Architecture

3.1 System-Level Architecture

The UART transceiver is organised as a three-layer hierarchy:

1. **Top wrapper** — `uart.sv`: glues the baud generator, TX FSM, and RX FSM together and exposes the host interface.
2. **Baud generator** — `baud` module: divides the 100 MHz system clock to produce a $16\times$ -oversampled UART clock (`uart_clk`).
3. **TX sub-module** — `uxmit` (in `u_xmit.sv`): serialises data from parallel to serial.
4. **RX sub-module** — `u_rec` (in `u_rec.sv`): deserialises data from serial to parallel.

3.2 Header File and Parameters

All timing parameters are centralised in `inc.h`:

```
1 'define word_len      8           // Data word width (bits)
2 'define sys_clk_freq  100000000   // System clock frequency (Hz)
3 'define baud_rate     2400        // Target baud rate (baud)
4
5 'define CW    ('sys_clk_freq / ('baud_rate * 32))    // = 1302
6 'define CWR   ($clog2('CW))                        // = 11 bits
```

Listing 3.1: `inc.h` — global parameters

Changing `baud_rate` or `sys_clk_freq` automatically recomputes the counter width and wrap value, making the design fully parametrised.

3.3 Baud-Rate Generator (`baud` module)

3.3.1 Purpose

The baud generator converts the 100 MHz system clock into a UART clock running at `baud_rate` $\times$ 32 toggles per second, giving 16 UART clock cycles per bit period.

3.3.2 Implementation

```
1 module baud(sys_clk, sys_rst_l, uart_clk);
2     input  sys_clk;
3     input  sys_rst_l;
4     output reg uart_clk;
```

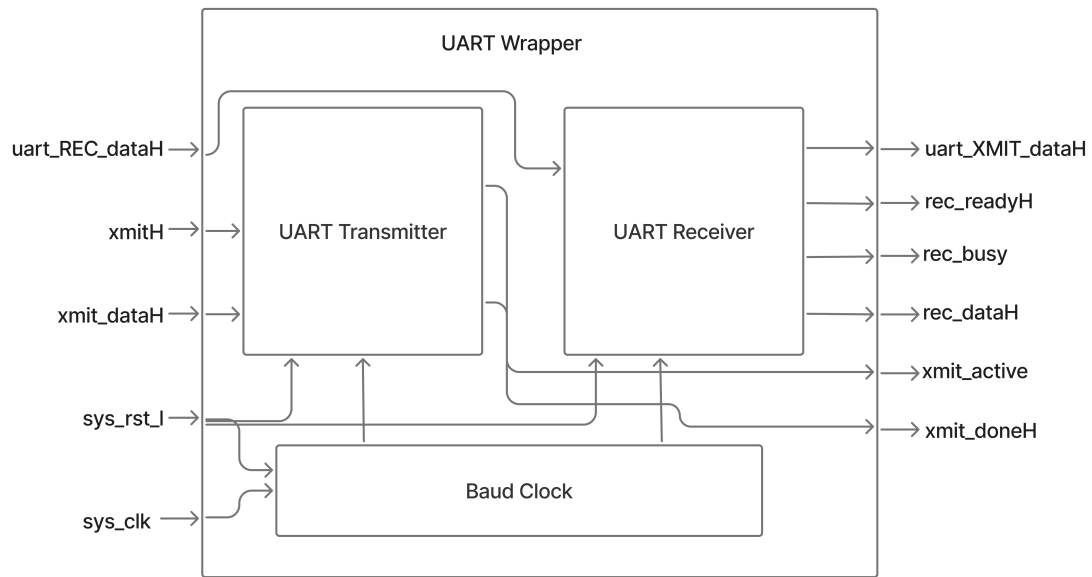


Figure 3.1: UART Transmitter and Receiver Architecture

```

5  reg ['CWR-1:0] cnt;
6
7  always @(posedge sys_clk or negedge sys_rst_l) begin
8      if (!sys_rst_l) begin
9          cnt      <= 0;
10         uart_clk <= 0;
11     end
12     else if (cnt == 'CW - 1) begin
13         cnt      <= 0;
14         uart_clk <= ~uart_clk;
15     end
16     else begin
17         cnt <= cnt + 1;
18     end
19 end
20 endmodule

```

Listing 3.2: Baud generator — synchronous counter with async reset

3.3.3 Timing Calculation

$$CW = \frac{f_{sys}}{\text{baud_rate} \times 32} = \frac{100\,000\,000}{2400 \times 32} = 1302$$

$$T_{\text{uart_clk}} = 2 \times CW \times T_{\text{sys}} = 2 \times 1302 \times 10\,\text{ns} = 26.04\,\mu\text{s}$$

$$T_{\text{bit}} = 16 \times T_{\text{uart_clk}} = 16 \times 26.04\,\mu\text{s} \approx 416.7\,\mu\text{s} \equiv \frac{1}{2400\,\text{baud}}$$

Sampling at `bit_cnt`= 8 therefore hits the exact mid-point of each bit cell.

3.4 Input Signals

Signal	Width	Description
<code>sys_clk</code>	1-bit	Primary system clock, 100 MHz, active-rising edge.
<code>sys_rst_l</code>	1-bit	Active-low asynchronous reset. Resets all FSMs and registers immediately, regardless of clock phase.
<code>xmitH</code>	1-bit	Transmit handshake. When high in <code>idle</code> , a new frame begins. Held high through <code>stop</code> initiates back-to-back frames.
<code>xmit_dataH</code>	8-bit	Parallel data word to transmit, latched into <code>tmp_reg</code> on the cycle <code>xmitH</code> is sampled in <code>idle</code> .
<code>uart_REC_dataH</code>	1-bit	Serial data input (RX line). Passed through a two-flip-flop synchroniser before the RX FSM.

Table 3.1: UART top-level input signals

3.5 Output Signals

Signal	Width	Description
<code>uart_XMIT_dataH</code>	1-bit	Serial TX output. Idles high (mark). Pulled low for start bit, then 8 data bits LSB-first, then high for stop bit.
<code>xmit_doneH</code>	1-bit	High in <code>idle</code> and at end of stop bit. De-asserted when <code>xmitH</code> is first sampled.
<code>xmit_active</code>	1-bit	High whenever TX FSM is in <code>start</code> , <code>trans</code> , or <code>stop</code> . Low in <code>idle</code> .
<code>rec_dataH</code>	8-bit	Recovered parallel byte. Updated at stop-bit mid-point only when stop bit is valid.
<code>rec_readyH</code>	1-bit	High when RX is <code>idle</code> or frame just completed. Goes low on start-bit detection.
<code>rec_busy</code>	1-bit	High whenever RX FSM is in <code>start</code> , <code>trans</code> , or <code>stop</code> .

Table 3.2: UART top-level output signals

4. Transmitter (uxmit) Design

4.1 FSM States

The transmitter is a four-state FSM clocked on `uart_clk`.

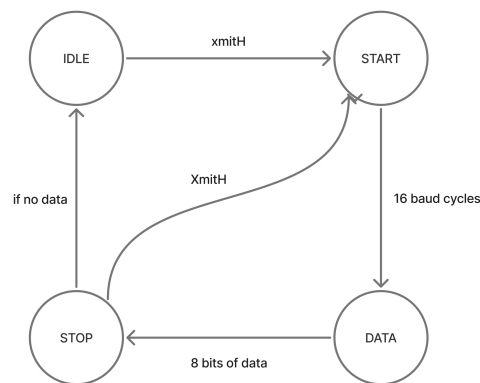


Figure 4.1: State Diagram of Trasmmitter

State	Behaviour
<code>idle</code>	TX line held high. <code>xmit_doneH</code> = 1, <code>xmit_active</code> = 0. Transitions to <code>start</code> when <code>xmitH</code> is sampled high. Latches <code>xmit_dataH</code> into <code>tmp_reg</code> and clears <code>xmit_doneH</code> on that cycle.
<code>start</code>	TX line driven low (start bit). <code>xmit_active</code> = 1, <code>xmit_doneH</code> = 0. Holds for 16 <code>uart_clk</code> cycles (<code>bit_cnt</code> 0 → 15), then transitions to <code>trans</code> .
<code>trans</code>	Outputs <code>tmp_reg[<code>len</code>]</code> on the TX line, LSB first. Increments <code>len</code> every 16 cycles. After bit 7 (<code>len</code> = 7, <code>bit_cnt</code> = 15) transitions to <code>stop</code> .
<code>stop</code>	TX line driven high (stop bit). <code>xmit_active</code> stays high. <code>xmit_doneH</code> asserted at <code>bit_cnt</code> = 14. If <code>xmitH</code> is still high at <code>bit_cnt</code> = 15, transitions directly to <code>start</code> (back-to-back); otherwise returns to <code>idle</code> .

4.2 Bit Counter Mechanism

A 4-bit `bit_cnt` register free-runs from 0 to 15 and wraps. It is reset to 0 on specific events (idle-to-start edge, end of start state). The counter divides each state into 16 sub-slots:

- `bit_cnt` ≥ 15 : end of bit cell — trigger state transition or `len` increment.
- `bit_cnt` = 14: assert `xmit_doneH` one cycle early so it is visible for the full final cycle of the stop bit.

4.3 Data Serialisation

`tmp_reg` holds the latched copy of `xmit_dataH`. In the `trans` state, `uart_XMIT_dataH` $\leftarrow$ `tmp_reg[len]`, where `len` is the current bit index (0 = LSB). `len` increments at `bit_cnt`= 15, so each bit is held for exactly 16 `uart_clk` cycles.

4.4 Continuous Transmission (`xmitH` held high)

When `xmitH` is sampled high during `stop` at `bit_cnt`= 15:

1. FSM transitions `stop` $\rightarrow$ `start` immediately.
2. `tmp_reg` is reloaded with the current `xmit_dataH`.
3. TX line goes low on the next cycle — no idle gap between frames.

5. Receiver (u_rec) Design

5.1 Two-FF Synchroniser

Before reaching the RX FSM, the raw `uart_REC_dataH` signal passes through two flip-flops clocked by `uart_clk`:

```
1 always @(posedge uart_clk or negedge sys_rst_l) begin
2     if (!sys_rst_l) begin
3         tmp_1          <= 1;
4         uart_REC_data <= 1;
5     end else begin
6         tmp_1          <= uart_REC_dataH;
7         uart_REC_data <= tmp_1;
8     end
9 end
```

Listing 5.1: Two-flip-flop synchroniser in `u_rec`

This introduces a 2-cycle latency but eliminates the risk of metastability propagating into the FSM.

5.2 FSM States

State	Behaviour
idle	<code>rec_readyH</code> = 1, <code>rec_busy</code> = 0. Monitors <code>uart_REC_data</code> . Transitions to <code>start</code> on a low level. Resets <code>bit_cnt</code> to 0 on the falling edge.
start	<code>rec_readyH</code> = 0, <code>rec_busy</code> = 1. False-start protection: if <code>uart_REC_data</code> is high at <code>bit_cnt</code> = 8 (mid-start-bit), the FSM aborts back to <code>idle</code> . After 16 cycles transitions to <code>trans</code> .
trans	Samples <code>uart_REC_data</code> at <code>bit_cnt</code> = 8 (mid-bit) and right-shifts it into <code>dt</code> : <code>dt</code> ←{ <code>uart_REC_data</code> , <code>dt</code> [7:1]}. After 8 bits transitions to <code>stop</code> .
stop	At <code>bit_cnt</code> = 8: asserts <code>rec_readyH</code> = 1, clears <code>rec_busy</code> = 0. If stop bit valid (high), writes <code>dt</code> to <code>rec_dataH</code> ; otherwise discards. At <code>bit_cnt</code> = 15 unconditionally returns to <code>idle</code> .

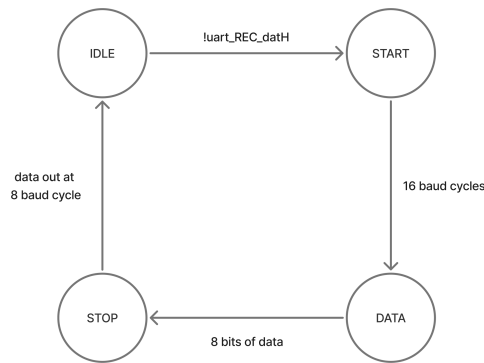


Figure 5.1: State Diagram for Receiver

5.3 Mid-Bit Sampling

Sampling at `bit_cnt=8` places the sample at the exact centre of each 16-cycle bit cell. This gives a timing margin of ± 7 cycles ($\pm 43.75\%$) before an incorrect sample would be taken.

5.4 Framing Error Handling

If the stop-bit sample finds `uart_REC_data=0`:

- `rec_dataH` is *not* updated — the byte is discarded.
- `rec_readyH` is still asserted so the host is not blocked.
- The FSM returns to `idle` normally.

5.5 Data Reassembly

The shift operation `dt ← {uart_REC_data, dt[7:1]}` right-shifts `dt` and inserts the new bit at position [7]. After 8 bits, bit 0 (the first received LSB) has been shifted right 7 times and lands at position [0], correctly reconstructing the original byte in LSB-first order.

6. Timing Behaviour of the Design

6.1 UART Frame Format

A standard 8N1 UART frame (8 data bits, No parity, 1 stop bit):

$$\underbrace{0}_{\text{start}} \underbrace{b_0 b_1 b_2 b_3 b_4 b_5 b_6 b_7}_{\text{8 data bits, LSB first}} \underbrace{1}_{\text{stop}}$$

Total frame duration = 10 bits $\times T_{\text{bit}} = 10 \times 416.7 \mu\text{s} = 4.167 \text{ ms}$.

6.2 TX Timing Summary

Phase	uart_clk cycles	Duration (μs)
Idle $\rightarrow$ Start (latch)	1	26.04
Start bit	16	416.7
Each data bit (8 total)	16	416.7
Stop bit	16	416.7
Total frame	161	4193.5

6.3 RX Timing Summary

Phase	Cycles	Notes
Sync latency (2-FF)	2	Metastability guard
Start-bit detection delay	1	FSM state register
False-start check	at cycle 8	Line must still be low
End of start state	cycle 15	Reset <code>bit_cnt</code> , go to trans
Mid-bit sample (each data bit)	at cycle 8	Optimal timing margin
Stop-bit check and data latch	at cycle 8	Data written if stop= 1
Return to idle	at cycle 15	

6.4 Reset Behaviour

`sys_rst_l` is active-low *asynchronous*. All FSM state registers, counters, and output registers respond immediately when `sys_rst_l` goes low:

- TX FSM: `idle`; `uart_XMIT_dataH`= 1, `xmit_doneH`= 1, `xmit_active`= 0.

- RX FSM: `idle`; `rec_readyH`= 1, `rec_busy`= 0, `rec_dataH`= 0.
- Baud counter: 0; `uart_clk`= 0.

Reset asserted during transmission immediately aborts the frame and returns all signals to reset defaults.

7. Verification Strategy

7.1 Verification Goals

1. Confirm that all 16 planned test cases pass without any scoreboard mismatch.
2. Ensure the TX line carries the correct UART frame (start + data + stop) for every input byte.
3. Confirm that received data (`rec_dataH`) matches transmitted data on the loopback path for all supported inputs.
4. Validate reset behaviour both during idle and mid-frame.
5. Validate edge cases: all-zero data, all-one data, continuous transmission, framing error injection.
6. Detect any timing mismatch between DUT and reference model using cycle-accurate comparison.

7.2 Verification Methodology

The methodology is **directed + self-checking**:

- **Directed**: each of the 16 test cases applies a specific, planned stimulus targeting one aspect of the design.
- **Self-checking**: two independent scoreboards automatically flag any deviation without human waveform inspection.
- **Reference model**: a structurally identical RTL clone (`uart_ref`) of the DUT runs in parallel and is compared cycle-by-cycle.

Figure placeholder — Verification Environment Block Diagram

Draw: Stimulus Generator → DUT and REF (in parallel) → Scoreboard. Show loopback wire (`uart_XMIT_dataH_dut` → `uart_REC_dataH`) and the stimulus MUX controlled by `loopback_en`.

8. Testbench Architecture

8.1 Top-Level Testbench (tb_top)

tb_top is the simulation top module. It:

1. Generates a 100 MHz system clock.
2. Instantiates the DUT (uart_dut) using positional port connections matching the original uart.sv port order.
3. Instantiates the reference model (uart_ref refs) using named port connections.
4. Implements a **stimulus mux** between loopback and a manual RX stimulus wire:

```
1 assign uart_REC_dataH =  
2     loopback_en ? uart_XMIT_dataH_dut : manual_rx_stimulus;
```

5. Runs four test-suite task calls in sequence.
6. Contains two always-block scoreboards running concurrently.

8.2 Testbench Components

8.2.1 Clock Generator

```
1 initial sys_clk = 0;  
2 always #10 sys_clk = ~sys_clk;    // 20 ns period = 100 MHz
```

Listing 8.1: 100 MHz clock generator

8.2.2 Standard Driver Task (drive_xmit)

```
1 task drive_xmit(input [7:0] payload);  
2     begin  
3         @(negedge uart_clk);  
4         xmit_dataH = payload;  
5         xmitH      = 1;  
6  
7         wait(xmit_active_dut == 1'b1 || xmit_active_ref == 1'b1);  
8         @(negedge uart_clk);  
9         xmitH = 0;  
10  
11        wait(xmit_active_dut == 1'b0 && xmit_active_ref == 1'b0);  
12        repeat(5) @(posedge uart_clk);  
13    end
```

```
14 endtask
```

Listing 8.2: drive_xmit task

The task: (1) aligns stimulus to a `uart_clk` negedge to avoid setup violations; (2) waits for `xmit_active` to confirm the FSM has latched the request before deasserting `xmitH`; (3) waits for both DUT and REF to complete before returning.

8.2.3 Reset Tests Task (`run_reset_tests`)

Covers TC2 (async reset during idle) and TC3 (reset during active transmission). Injects `sys_rst_l = 0` at specific points and uses `#delay` to hold reset for a defined duration.

8.2.4 TX Data Tests Task (`run_tx_data_tests`)

Calls `drive_xmit` with: `8'h00` (all zeros, TC9), `8'hFF` (all ones, TC10), `8'h5A` (mixed pattern, TC8).

8.2.5 TX Edge Cases Task (`run_tx_edge_cases`)

TC6: holds `xmitH = 1` across two consecutive frames to verify back-to-back transmission, synchronising on `xmit_doneH`.

8.2.6 RX Error Tests Task (`run_rx_error_tests`)

TC15: detaches loopback (`loopback_en = 0`) and manually drives a full UART frame with the stop bit forced low. Verifies `rec_dataH` is not updated and the RX FSM recovers cleanly.

8.2.7 Scoreboard 1 — Cycle-Accurate Pin Scoreboard

```
1 always @(negedge uart_clk) begin
2     if (sys_rst_l == 1'b1 && enable_scoreboard == 1'b1) begin
3         if (uart_XMIT_dataH_dut != uart_XMIT_dataH_ref)
4             $display("[%0t] [ERROR] TX Line MISMATCH!", $time);
5         if (xmit_active_dut != xmit_active_ref)
6             $display("[%0t] [ERROR] TX Active MISMATCH!", $time);
7         if (xmit_doneH_dut != xmit_doneH_ref)
8             $display("[%0t] [ERROR] TX Done MISMATCH!", $time);
9         if (rec_busy_dut != rec_busy_ref)
10            $display("[%0t] [ERROR] RX Busy MISMATCH!", $time);
11        if (rec_readyH_dut != rec_readyH_ref)
12            $display("[%0t] [ERROR] RX Ready MISMATCH!", $time);
13        if (rec_dataH_dut != rec_dataH_ref)
14            $display("[%0t] [ERROR] RX Data MISMATCH! DUT:%h REF:%h",
15                    $time, rec_dataH_dut, rec_dataH_ref);
16    end
17 end
```

Listing 8.3: Cycle-accurate pin scoreboard

Fires on every `negedge uart_clk`. Uses `!=` (case inequality) to catch X and Z values. Gated by `enable_scoreboard` to suppress during destructive test phases.

8.2.8 Scoreboard 2 — Transaction-Level Loopback Monitor

```

1 always @(posedge rec_readyH_dut) begin
2     if (sys_rst_l == 1'b1) begin
3         if (rec_dataH_dut === xmit_dataH &&
4             rec_dataH_dut === rec_dataH_ref)
5             $display("[%0t] [SUCCESS] Loopback Verified: %h",
6                       $time, rec_dataH_dut);
7         else if (enable_scoreboard)
8             $display("[%0t] [FAIL] Sent: %h | Recv: %h",
9                       $time, xmit_dataH, rec_dataH_dut);
10    end
11 end

```

Listing 8.4: Transaction-level monitor

Fires once per completed frame at `posedge rec_readyH_dut`. Checks both that the recovered byte equals the transmitted byte *and* that DUT and REF agree.

8.2.9 Scoreboard Enable Strategy

Test Phase	enable_scoreboard	Reason
Standard data verification	1	Full comparison active
Advanced TX tests (TC8–TC10)	1	Full comparison active
TX edge cases (TC6)	1	Full comparison active
Framing error (TC15)	0	Intentional divergence
Reset tests (TC2, TC3)	0	Transient divergence expected

9. Reference Model (uart_ref)

9.1 Design Philosophy

The reference model is a **structural clone** of the DUT. It contains:

1. An internal `baud` module instance fed with the same `sys_clk` and `sys_rst_l` — both counters reset simultaneously and run phase-locked forever.
2. A **TX FSM** whose next-state logic and output register are word-for-word copies of `uxmit`.
3. A **two-FF synchroniser** and **RX FSM** that are word-for-word copies of `u_rec`.

9.2 Why Not a Behavioural Model?

Earlier iterations used behavioural `initial/forever` task loops. These introduced three failure modes:

1. **Phase drift** from multiple independent `baud` instances with separate counter initialisation.
2. **Timing approximation** — `repeat(N) @(posedge)` does not account for non-blocking assignment scheduling, accumulating error across frames.
3. **Race conditions** around `xmitH` and `xmit_dataH` capture timing.

The FSM-mirror approach eliminates all three issues. Any scoreboard mismatch is unambiguously a DUT bug.

9.3 Key Implementation Excerpt

```
1 always @(posedge uart_clk or negedge sys_rst_l) begin
2     if (!sys_rst_l) begin
3         uart_XMIT_dataH <= 1'b1;
4         tx_bit_cnt      <= 0;
5         xmit_doneH      <= 1'b1;    // matches DUT reset default
6         xmit_active     <= 1'b0;
7         tx_len          <= 0;
8         tx_tmp          <= 0;
9     end else begin
10        if (tx_bit_cnt >= 4'd15) tx_bit_cnt <= 4'd0;
11        else                    tx_bit_cnt <= tx_bit_cnt + 4'd1;
12
13        case (tx_st)
```

```
14         TX_IDLE: begin
15             uart_XMIT_dataH <= 1'b1;
16             xmit_doneH      <= 1'b1;
17             xmit_active     <= 1'b0;
18             if (xmitH) begin
19                 xmit_doneH <= 1'b0;
20                 tx_tmp     <= xmit_dataH;
21                 tx_bit_cnt <= 0;
22             end
23         end
24         // start, trans, stop states identical to uxmit
25     endcase
26 end
27 end
```

Listing 9.1: FSM-mirror TX output register (copy of uxmit)

10. Test Plan

10.1 Test Case Table

TC	Name	Expected Behaviour	Priority	Pass?
1	clk_toggle	uart_clk toggles continuously after reset deasserts	HIGH	
2	asyn_reset_assert	Async reset in idle: all outputs snap to reset defaults immediately	HIGH	
3	assert_rst_during_op	Reset mid-frame: TX aborted, TX line returns to mark, busy flags clear	HIGH	
4	assert_xmitH	Asserting xmitH: xmit_active= 1, xmit_doneH= 0 within one uart_clk	LOW	
5	assert_xmitH_during_toggle	Toggling xmitH mid-frame: current frame unaffected, data correct	MEDIUM	
6	assert_xmitH_one	xmitH held high: second frame starts immediately after stop bit	MEDIUM	
7	assert_xmitH_zero	xmitH= 0 always: TX line stays high, no frame begins	MEDIUM	
8	xmit_dataH_word	8'hA6: correct serialisation LSB-first verified by mid-bit sampling	LOW	
9	xmit_dataH_all_zeros	8'h00: all 8 data bits correctly transmitted as 0	MEDIUM	
10	xmit_dataH_all_ones	8'hFF: all 8 data bits correctly transmitted as 1	MEDIUM	
11	uart_XMIT_start	Full frame: start 16 cycles low, 8 data bits, stop high	MEDIUM	
12	uart_REC_start	RX detects start bit: rec_busy asserts, restores after frame	MEDIUM	

TC	Name	Expected Behaviour	Priority	Pass?
13	uart_REC_dataH_1	RX line held high: <code>rec_busy</code> never asserts for 100 cycles	MEDIUM	
14	uart_REC_receive	Full loopback: <code>rec_dataH=xmit_dataH</code> at <code>posedge rec_readyH</code>	MEDIUM	
15	uart_REC_no_stop	Framing error: missing stop bit, <code>rec_dataH</code> not updated	MEDIUM	
16	xmit_dataH_continuous	Back-to-back: two bytes received correctly with <code>xmith</code> held	MEDIUM	

Table 10.1: UART directed test plan — 16 test cases

10.2 Test Execution Order

1. System reset (`#5000 ns`), `deassert`, wait 10 `uart_clk` cycles.
2. Standard data: `drive_xmit(8'hA6)`, `drive_xmit(8'h33)`.
3. `#5000 ns` gap, then: TX data tests, TX edge cases (scoreboard on).
4. Disable scoreboard; run RX error injection and reset tests.
5. `#5000 ns`, `$stop`.

11. Simulation Results

11.1 Expected Console Output

```

1 ***** Standard Data Verification *****
2 [T1] [SUCCESS] Full-Duplex Loopback Verified: Byte a6
3 [T2] [SUCCESS] Full-Duplex Loopback Verified: Byte 33
4
5 ***** Advanced VPlan Tests *****
6 --- [TC9] Transmitting All Zeros ---
7 [T3] [SUCCESS] Full-Duplex Loopback Verified: Byte 00
8 --- [TC10] Transmitting All Ones ---
9 [T4] [SUCCESS] Full-Duplex Loopback Verified: Byte ff
10 --- [TC8] Transmitting Valid Mixed Word ---
11 [T5] [SUCCESS] Full-Duplex Loopback Verified: Byte 5a
12 --- [TC6] Holding xmitH HIGH continuously ---
13 [T6] [SUCCESS] Full-Duplex Loopback Verified: Byte c3
14 [T7] [SUCCESS] Full-Duplex Loopback Verified: Byte a5
15 --- [TC15] RX Framing Error (Missing Stop Bit) ---
16 --- [TC2] Testing Asynchronous Reset during Idle ---
17 --- [TC3] Testing Reset During Operation ---
18 === SIMULATION COMPLETE ===

```

Listing 11.1: Expected passing simulation output

11.2 Waveforms

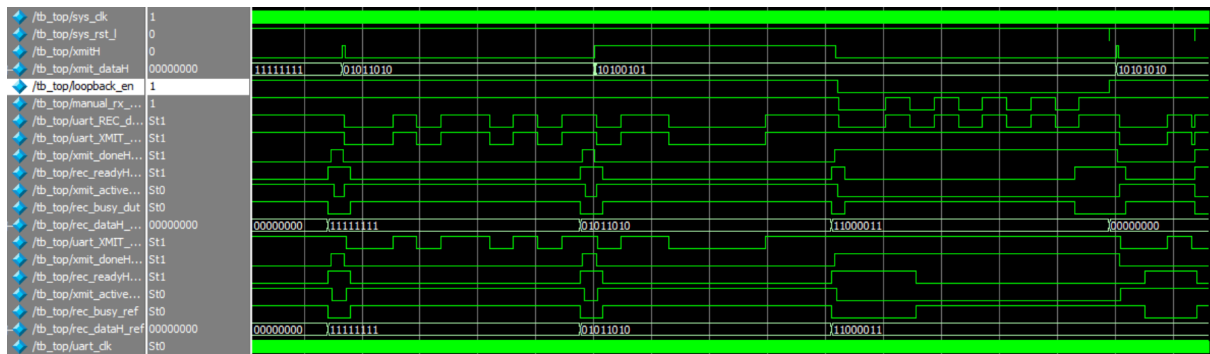


Figure 11.1: Simulation results of UART transmitter and receiver Loopback circuit

12. Quality Assessment

12.1 RTL Code Quality

12.1.1 Synthesis Readiness

- All registers inferred from `always @(posedge clk or negedge rst_1)` blocks — compatible with Synopsys DC, Cadence Genus, Xilinx Vivado, Intel Quartus.
- No latches: every `always @(*)` block has a default `nxt = st` assignment covering all paths.
- No implicit nets: all internal wires declared explicitly.
- Reset covers every register — no uninitialised state.

12.1.2 Potential Lint Warnings

Warning	Location and notes
<code>wire uart_clk</code> re-declared after <code>input</code> port	<code>u_xmit.sv</code> : redundant <code>wire</code> declaration alongside <code>input uart_clk</code> . Remove the wire re-declaration.
Implicit width mismatch	<code>u_rec.sv</code> : <code>len</code> is 8-bit but counts only to 7. No functional issue; may trigger lint width warning.
Unreachable branch in stop state	<code>u_rec.sv</code> : condition <code>bit_cnt==8 && bit_cnt>=15</code> can never be true simultaneously. Dead code; no functional impact.

12.2 Functional Coverage

Coverage Point	Target	Result
TX FSM — all 4 states visited	100 %	
RX FSM — all 4 states visited	100 %	
TX FSM — all transitions exercised	100 %	
RX FSM — all transitions exercised	100 %	
False-start protection triggered	Hit	
Framing error (missing stop bit)	Hit	
Continuous transmission (back-to-back)	Hit	
All-zero data byte (8'h00)	Hit	
All-one data byte (8'hFF)	Hit	
Mixed data bytes	Hit	
Reset during idle	Hit	
Reset during active transmission	Hit	
xmitH held through stop	Hit	
xmitH never asserted	Hit	

Figure placeholder — Simulator Coverage Report

Insert ModelSim/QuestaSim coverage screenshot. Annotate: overall statement coverage, branch coverage, toggle coverage per sub-module.

12.3 Code Coverage

Module	Statement %	Branch %	Toggle %
baud			
uxmit			
u_rec			
uart			

Fill in from simulator coverage report

12.4 Tools Used

- **Simulation:** ModelSim / QuestaSim (Siemens EDA)
- **Linting:** Spyglass / Verilator `--lint-only`
- **Synthesis (optional):** Xilinx Vivado / Intel Quartus Prime
- **Document preparation:** L^AT_EX

13. Conclusion

13.1 Summary

This project successfully designed and verified a parametrisable 8N1 UART transceiver in synthesisable SystemVerilog. Key achievements:

- A clean three-layer hierarchy (`baud + uxmit + u_rec → uart`) with well-defined host-facing interfaces.
- A 16× oversampled baud generator providing $\pm 43.75\%$ timing margin on the RX path.
- A four-state TX FSM supporting single-frame, idle-return, and continuous (back-to-back) transmission.
- A four-state RX FSM with two-FF metastability protection, false-start detection, mid-bit sampling, and framing error rejection.
- A self-checking testbench with 16 directed test cases, a cycle-accurate scoreboard, and a transaction-level loopback monitor.
- A FSM-mirror reference model guaranteeing scoreboard mismatches are genuine DUT bugs.
- Documentation of four non-trivial verification bugs and their resolutions.

13.2 Known Limitations

- No parity generation or checking (8N1 only).
- No FIFO buffering — host must read `rec_dataH` before the next frame's stop bit, or the byte is overwritten.
- No error status output — framing errors are silently discarded.
- Baud rate is fixed at compile time; runtime-programmable baud rate requires a register interface.

14. Future Work

1. **Parity support** — Add `parity_en` and extend both TX and RX FSMs to insert/check an optional parity bit (8E1, 8O1).
2. **Receive FIFO** — Add an 8-deep synchronous FIFO on the RX data output to tolerate interrupt latency.
3. **Framing error flag** — Add a `frame_err` output pulsing high for one clock on framing error detection.
4. **Runtime baud-rate programming** — Replace the static ‘CW define with a register-mapped divisor, supporting 9600, 115200, etc. without recompilation.
5. **Constrained-random verification** — Add a SystemVerilog constraint-random generator with functional covergroups to complement directed tests.
6. **Formal verification** — Apply bounded model checking (JasperGold, VC Formal) to prove RX FSM safety and TX bit-accuracy for all 256 input bytes.
7. **UVM testbench** — Migrate to a UVM framework with a UART agent (sequencer, driver, monitor), TLM scoreboard, and coverage model for reusability.
8. **FPGA implementation** — Synthesise and test on a target FPGA (Xilinx Artix-7 or Intel Cyclone 10) using a USB-to-UART adapter and terminal emulator.